نظم التشغيل 2211431 · جامعة مؤتة · د. وفاء الطراونة · فريق Kernel Crew

CPU SchedLab

دليل الشرح لأعضاء الفريق

لكل عضو: صورة جزئيته من الكود، وتحتها فقرة جاهزة يقولها بصوته للدكتورة.

ملف الكود

المشروع كله في ملف واحد اسمه **CPU_SchedLab.py**، ومقسّم إلى 5 أجزاء، وكل جزء مكتوب فوقه اسم صاحبه (PART 1 … PART 5). أرقام الأسطر في الصور هي نفس أرقام الأسطر في الملف. افتح الملف في VS Code واضغط **Ctrl + G** واكتب رقم السطر للانتقال إليه.

من يشرح ماذا

#	العضو	أسطره في الملف	الوظائف
1	**محمد مصطفى المكاوي** قائد الفريق · المحرّك الأساسي و FCFS	34–50 83–99 102–113 342–360	F1 · F2 · F3
2	**سامي عرفات المعايطة** SJF و SRTF	121–138 141–160	F4 · F5
3	**محمد عبد الإله الدلاعين** المقارنة · Round Robin · Context switch	168–197 200–231 237–243	F6 · F10 · F11
4	**عروب أحمد العمرو** المقاييس · Priority scheduling	275–307 310–334	F7 · F9
5	**سارة محمد المجالي** الواجهة الرسومية والرسم	375–382 402–415 725–742 466–491	F8 · F12

طريقة التشغيل (على ويندوز)

1

ثبّت Python 3.12 من python.org

2

في الـ Terminal اكتب:

pip install matplotlib

3

شغّل البرنامج:

python CPU_SchedLab.py

المثال الذي سنستخدمه كلنا (صفحة 3 في المقترح)

العملية	وقت الوصول	Burst	الأولوية
P1	0	5	2
P2	1	3	1
P3	2	1	3

متوسط الانتظار: **Round Robin (q=2) = 3.33** · **SRTF = 1.67** · **SJF = 2.67** · **FCFS = 3.33**. وهذا المثال يفتح تلقائياً عند تشغيل البرنامج. وباقي الأمثلة موجودة في قائمة **Examples** داخل البرنامج.

نصيحة للفيديو: عرّف بنفسك (10 ثوانٍ) ← اشرح الفكرة (30 ثانية) ← اقرأ فقرة الكود وأنت تؤشر على الأسطر (90 ثانية) ← اعرض البرنامج وهو يشتغل (40 ثانية) ← خلاصة بجملة واحدة.

1 محمد مصطفى المكاوي

قائد الفريق · المحرّك الأساسي و F3 · F2 · F1 · FCFS

🎤 البداية — أعرّف بنفسي

السلام عليكم دكتورة، أنا محمد المكاوي قائد الفريق. أنا مسؤول عن الجزء الأول من المشروع: تمثيل العملية (PCB)، وإدخال العمليات، والتوليد العشوائي، وخوارزمية FCFS، والقلب (core) الذي يربط أجزاء الفريق كلها ببعض.

الجزء 1 — العملية Process (الـ PCB)

CPU_SchedLab.py — lines 34–50

```python
@dataclass
class Process:
    """One process, like a small PCB. Smaller priority number = more important."""
    pid: str
    arrival: int
    burst: int
    priority: int = 0

    def __post_init__(self):
        if not str(self.pid).strip():
            raise ValueError("Process id cannot be empty.")
        if self.arrival < 0:
            raise ValueError(f"{self.pid}: arrival time cannot be negative.")
        if self.burst <= 0:
            raise ValueError(f"{self.pid}: burst time must be greater than 0.")
        if self.priority < 0:
            raise ValueError(f"{self.pid}: priority cannot be negative.")
```

🎤 ماذا أقول للدكتورة

هنا عرّفت الكلاس **Process**، وهو يمثّل العملية داخل نظام التشغيل بشكل مبسّط، يعني نسخة مصغّرة من الـ **PCB**. كل عملية فيها أربع معلومات: اسمها **pid**، ووقت وصولها إلى الـ ready queue وهو **arrival**، والوقت الذي تحتاجه من المعالج وهو **burst**، وأولويتها **priority**، وكلما كان رقم الأولوية أصغر كانت العملية أهم، مثل الكتاب. وفي الدالة __post_init__ أتحقق من البيانات لحظة إنشاء العملية: لا أسمح باسم فارغ، ولا بوقت وصول سالب، ولا بـ burst يساوي صفراً، وأي خطأ يظهر للمستخدم برسالة واضحة، وبهذا حققنا متطلب **Reliable** الموجود في المقترح.

الجزء 2 — إدخال العمليات والتوليد العشوائي (F1 · F2)

CPU_SchedLab.py lines 83–99

```python
def make_process(pid, arrival, burst, priority="0"):
    """Turn the text typed by the user into a Process, with a clear error message."""
    numbers = []
    for name, text in (("Arrival", arrival), ("Burst", burst), ("Priority", priority)):
        text = str(text).strip()
        if not text.lstrip("-").isdigit():
            raise ValueError(f"{name} must be a whole number (you typed '{text}').")
        numbers.append(int(text))
    return Process(str(pid).strip(), *numbers)


def random_workload(count=6, max_arrival=10, max_burst=10, max_priority=5):
    """F2: generate `count` random processes P1..Pn for quick testing."""
    if not 1 <= count <= 50:
        raise ValueError("Number of processes must be between 1 and 50.")
    return [Process(f"P{i}", random.randint(0, max_arrival), random.randint(1, max_burst),
                    random.randint(0, max_priority)) for i in range(1, count + 1)]
```

🎤 ماذا أقول للدكتورة

الدالة **make_process** تأخذ النص الذي يكتبه المستخدم في الواجهة وتحوّله إلى عملية، وإذا كتب حرفاً بدل رقم تخبره بالضبط ما هو الخطأ. أما **random_workload** فهي الوظيفة **F2**: تولّد من عملية واحدة حتى خمسين عملية بقيم عشوائية لوقت الوصول والـ burst والأولوية. هذا يمثّل مفهوم **Process creation**، ويساعدنا نختبر الخوارزميات بسرعة على حالات كثيرة بدل ما ندخلها يدوياً.

الجزء 3 — خوارزمية FCFS (F3)

CPU_SchedLab.py lines 102–113

```python
def fcfs_schedule(processes, options):
    """F3 FCFS: the first to ARRIVE runs first, until it finishes (non-preemptive)."""
    cpu = Dispatcher(options.switch_cost)
    time = 0
    for p in sorted(processes, key=lambda p: p.arrival):   # ready queue = FIFO by arrival
        if time < p.arrival:                                # nobody here yet -> CPU idle
            cpu.idle(time, p.arrival)
            time = p.arrival
        time = cpu.dispatch(p.pid, time)                    # give it the CPU
        cpu.run(p.pid, time, time + p.burst)                # it runs its WHOLE burst
        time += p.burst
    return cpu.result("FCFS", processes)
```

🎤 ماذا أقول للدكتورة

هذه خوارزمية **FCFS**، أي من يصل أولاً يُخدم أولاً. أولاً أرتّب العمليات حسب وقت الوصول، وهذا هو الـ **ready queue** بنظام **FIFO**. ثم أمرّ عليها واحدة واحدة: إذا كان المعالج فارغاً والعملية لم تصل بعد، يبقى المعالج **idle** حتى تصل. بعدها أعطيها المعالج عن طريق الـ dispatcher، وتشتغل الـ burst كاملاً بدون أن يقاطعها أحد، ولهذا FCFS خوارزمية **non-preemptive**. في مثال المقترح: P1 من 0 إلى 5، ثم P2 حتى 8، ثم P3 حتى 9، ومتوسط الانتظار **3.33**. عيبها ما يسمى **Convoy effect**: إذا جاءت عملية طويلة في البداية، تنتظرها كل العمليات القصيرة.

الجزء 4 — القلب (Scheduler core) الذي يربط أجزاء الفريق

CPU_SchedLab.py lines 342–360

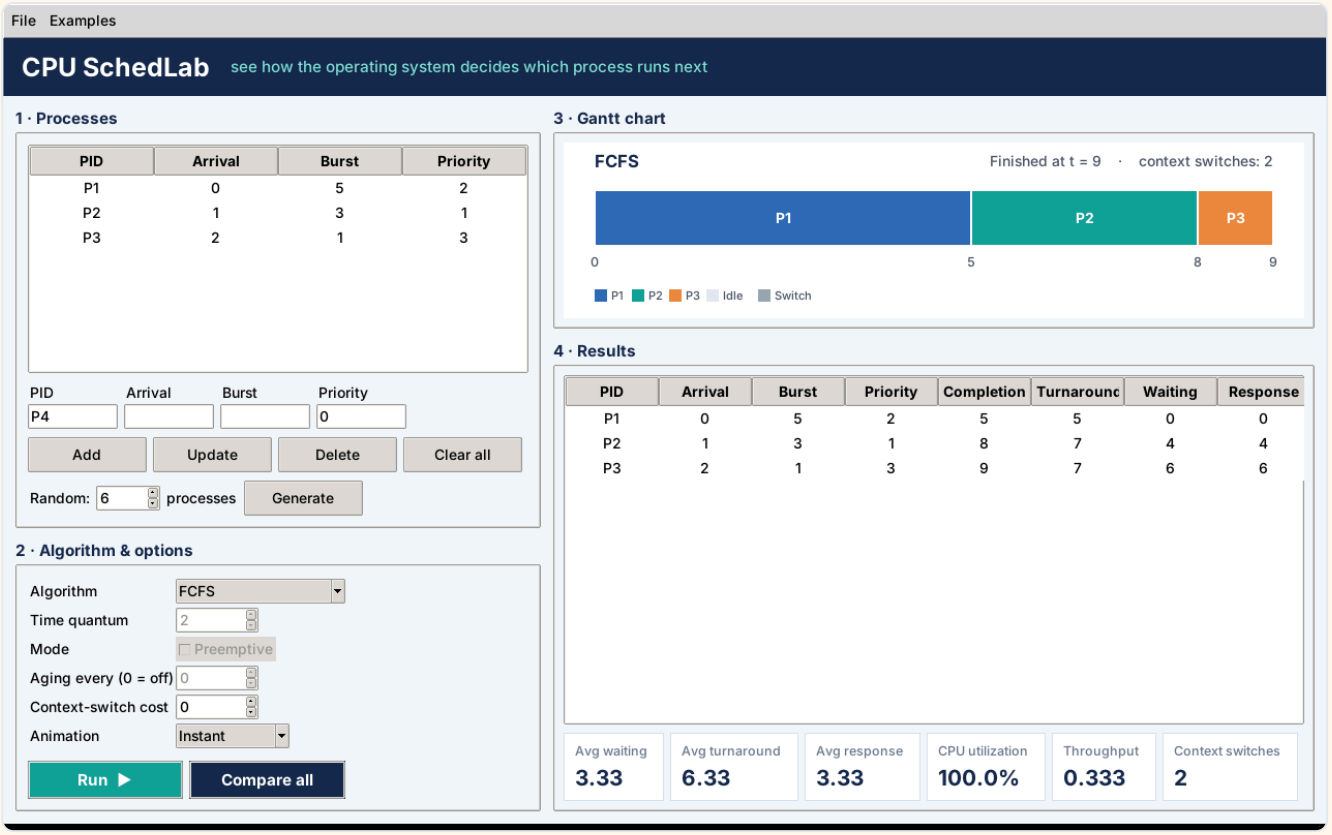
```python
ALGORITHMS = {
    "FCFS": fcfs_schedule,
    "SJF": sjf_schedule,
    "SRTF": srtf_schedule,
    "Round Robin": round_robin_schedule,
    "Priority": priority_schedule,
}


def run(algorithm, processes, options):
    """Check the input, pick the algorithm, run it on a COPY of the processes."""
    if not processes:
        raise ValueError("Add at least one process first.")
    ids = [p.pid for p in processes]
    if len(ids) != len(set(ids)):
        raise ValueError("Process ids must be unique.")
    if options.quantum <= 0 or options.aging < 0 or options.switch_cost < 0:
        raise ValueError("Quantum must be > 0; aging and switch cost cannot be negative.")
    return ALGORITHMS[algorithm](copy.deepcopy(processes), options)
```

🎤 ماذا أقول للدكتورة

هذا قلب البرنامج الذي يجمع عمل الفريق. القاموس **ALGORITHMS** يربط اسم كل خوارزمية بالدالة الخاصة بها، وكل الدوال لها نفس الشكل تماماً: تأخذ العمليات والخيارات وترجع **Result** فيه الخط الزمني. والدالة **run** هي الوحيدة التي تستدعيها الواجهة: تتأكد أن هناك عمليات، وأن الأسماء غير مكررة، وأن الخيارات صحيحة، ثم تشغّل الخوارزمية على نسخة **deepcopy** من العمليات حتى لا تتغيّر بيانات المستخدم. بفضل هذا التصميم كل عضو كتب واختبر جزأه وحده، وأنا جمعت الأجزاء بسطر واحد لكل خوارزمية.

🖥️ ماذا أعرض على الشاشة



أفتح البرنامج وأختار FCFS وأضغط Run. يظهر الـ Gantt chart: P1 من 0 إلى 5، P2 حتى 8، P3 حتى 9، ومتوسط الانتظار **3.33**، وهو نفس الرقم الموجود في المقترح. وأضغط Generate لأعرض التوليد العشوائي.

2 سامي عرفات المعايطة

SJF و F5 · F4 · SRTF

🎤 البداية — أعرّف بنفسي

السلام عليكم دكتورة، أنا سامي المعايطة، ومسؤول عن خوارزميتَي **SJF** و **SRTF**، وفكرتهما المشتركة: نعطي المعالج للعملية الأقصر.

الجزء 1 — خوارزمية SJF (F4)

CPU_SchedLab.py — lines 121–138

```python
def sjf_schedule(processes, options):
    """F4 SJF: when the CPU is free, run the ARRIVED process with the SHORTEST burst."""
    cpu = Dispatcher(options.switch_cost)
    time = 0
    not_started = list(processes)
    while not_started:
        ready = [p for p in not_started if p.arrival <= time]
        if not ready:                                   # nobody ready -> idle
            next_arrival = min(p.arrival for p in not_started)
            cpu.idle(time, next_arrival)
            time = next_arrival
            continue
        p = min(ready, key=lambda p: (p.burst, p.arrival))  # shortest burst wins
        time = cpu.dispatch(p.pid, time)
        cpu.run(p.pid, time, time + p.burst)                # runs to the end
        time += p.burst
        not_started.remove(p)
    return cpu.result("SJF", processes)
```

🎤 ماذا أقول للدكتورة

في خوارزمية **SJF**، كلما فرغ المعالج أختار من العمليات التي وصلت فعلاً العملية صاحبة **أقصر burst**. القائمة **ready** هي العمليات التي وصل وقتها ولم تشتغل بعد. إذا لم تكن هناك عملية جاهزة يبقى المعالج idle حتى أقرب وصول. الاختيار يتم بالدالة **min** على المفتاح (burst, arrival)، أي الأقصر أولاً، وإذا تساوتا فالأسبق وصولاً. وبعد اختيارها تشتغل حتى النهاية لأن SJF **non-preemptive**. في مثالنا، عند الزمن 5 كانت P2 تحتاج 3 و P3 تحتاج 1، فاخترنا P3، وأصبح المتوسط **2.67** بدلاً من 3.33. SJF تعطي **أقل متوسط انتظار ممكن**، لكن مشكلتها أننا في الواقع لا نعرف طول الـ burst مسبقاً، فالنظام يتوقعه باستخدام **exponential averaging**.

الجزء 2 — خوارزمية SRTF (F5)

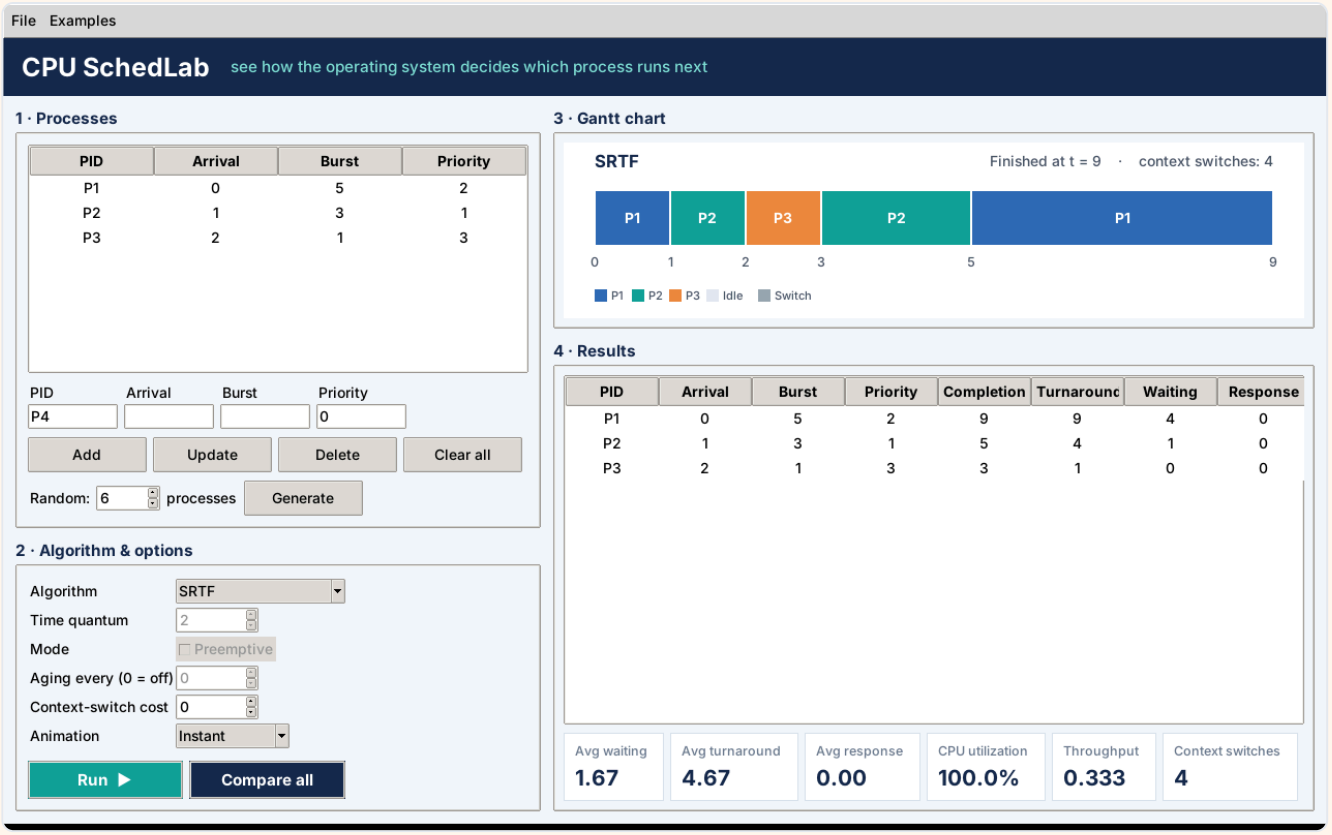
CPU_SchedLab.py lines 141–160

```python
def srtf_schedule(processes, options):
    """F5 SRTF: every time unit, run the process with the smallest REMAINING time."""
    cpu = Dispatcher(options.switch_cost)
    remaining = {p.pid: p.burst for p in processes}
    time, running = 0, None
    while any(remaining.values()):
        ready = [p for p in processes if p.arrival <= time and remaining[p.pid] > 0]
        if not ready:
            next_arrival = min(p.arrival for p in processes if remaining[p.pid] > 0)
            cpu.idle(time, next_arrival)
            time = next_arrival
            continue
        # smallest remaining time; on a tie keep the running one (no useless switch)
        p = min(ready, key=lambda p: (remaining[p.pid], p.pid != running, p.arrival))
        time = cpu.dispatch(p.pid, time)
        cpu.run(p.pid, time, time + 1)                    # ONE unit, then decide again
        time += 1
        remaining[p.pid] -= 1
        running = p.pid
    return cpu.result("SRTF", processes)
```

🎤 ماذا أقول للدكتورة

SRTF هي النسخة القابلة للمقاطعة من SJF. أحتفظ بقاموس **remaining** فيه الوقت المتبقي لكل عملية. في **كل وحدة زمنية** أعيد القرار: أختار العملية ذات أقل وقت متبقٍّ، وإذا تساوت مع العملية الحالية أُبقي الحالية حتى لا يحصل context switch بلا فائدة. ثم أشغّلها وحدة زمنية واحدة فقط وأرجع أقرر من جديد، وهذا بالضبط ما يجعلها **preemptive**. في المثال: عند الزمن 1 وصلت P2 وتحتاج 3، وهذا أقل من 4 المتبقية لـ P1، فقاطعتها. وعند الزمن 2 وصلت P3 وتحتاج 1 فقاطعت P2. النتيجة متوسط انتظار **1.67** وهو الأفضل، لكن الثمن 4 عمليات context switch، وقد تتعرض العمليات الطويلة للـ **starvation**.

🖥️ ماذا أعرض على الشاشة



أختار SRTF وأضغط Run. الرسم يوضح المقاطعات: P1 ثم P2 ثم P3 ثم P2 ثم P1، وعدد الـ **4** context switches، ومتوسط الانتظار **1.67**. ثم أرجع إلى SJF لأقارن: 2.67.

3 محمد عبد الإله الدلاعين

F6 · F10 · F11 · المقارنة · Round Robin · Context switch

🎤 البداية — أعرّف بنفسي

السلام عليكم دكتورة، أنا محمد الدلاعين، ومسؤول عن خوارزمية **Round Robin**، وعن حساب تكلفة الـ **context switch**، وعن **مقارنة** كل الخوارزميات بالرسوم.

الجزء 1 — خوارزمية Round Robin (F6)

CPU_SchedLab.py — lines 168–197

```python
def round_robin_schedule(processes, options):
    """F6 Round Robin: each process runs at most `quantum`, then goes to the back."""
    cpu = Dispatcher(options.switch_cost)
    arrivals = sorted(processes, key=lambda p: p.arrival)
    remaining = {p.pid: p.burst for p in processes}
    queue = deque()                                   # the ready queue (FIFO)
    time = 0

    def admit(up_to):                                 # arrived processes join the queue
        while arrivals and arrivals[0].arrival <= up_to:
            queue.append(arrivals.pop(0))

    admit(time)
    while queue or arrivals:
        if not queue:                                 # queue empty -> idle
            cpu.idle(time, arrivals[0].arrival)
            time = arrivals[0].arrival
            admit(time)
            continue
        p = queue.popleft()                           # take the FRONT process
        time = cpu.dispatch(p.pid, time)
        admit(time)
        run_for = min(options.quantum, remaining[p.pid])    # one quantum (or less)
        cpu.run(p.pid, time, time + run_for)
        time += run_for
        remaining[p.pid] -= run_for
        admit(time)                                   # new arrivals enter first
        if remaining[p.pid] > 0:
            queue.append(p)                           # not finished -> back of queue
    return cpu.result(f"Round Robin (q={options.quantum})", processes)
```

🎤 ماذا أقول للدكتورة

خوارزمية **Round Robin** تمثّل مفهوم الـ **time sharing**. الـ ready queue هنا طابور حقيقي **deque** بنظام FIFO. الدالة **admit** تُدخل إلى الطابور كل عملية وصل وقتها. في كل دورة آخذ العملية من **مقدمة** الطابور وأشغّلها مدة **quantum**، أو أقل إذا كان المتبقي لها أقل. إذا لم تنتهِ ترجع إلى **آخر** الطابور، مع ملاحظة أن العمليات التي وصلت للتو تدخل قبلها. جرّبناها على مثال الكتاب مع quantum يساوي 4 فكان متوسط الانتظار **5.67**، وهو نفس جواب الكتاب. إذا كان الـ quantum كبيراً جداً تصبح مثل FCFS، وإذا كان صغيراً جداً يضيع الوقت في الـ context switch.

الجزء 2 — الـ Dispatcher وتكلفة الـ Context switch (F11)

CPU_SchedLab.py lines 200–231

```python
class Dispatcher:
    """F11 Builds the timeline for EVERY algorithm and handles context switches."""

    def __init__(self, switch_cost=0):
        self.switch_cost = switch_cost
        self.timeline = []
        self.switches = 0
        self.last_pid = None

    def dispatch(self, pid, now):
        """Give the CPU to `pid`. A different process than before = context switch."""
        if self.last_pid is not None and pid != self.last_pid:
            self.switches += 1                              # save old PCB, load new PCB
            if self.switch_cost > 0:
                self.add(SWITCH, now, now + self.switch_cost)
                now += self.switch_cost                     # time lost (dispatcher latency)
        self.last_pid = pid
        return now                                          # when the process really starts

    def run(self, pid, start, end):
        self.add(pid, start, end)

    def idle(self, start, end):
        if end > start:
            self.add(IDLE, start, end)

    def add(self, pid, start, end):
        last = self.timeline[-1] if self.timeline else None
        if last and last.pid == pid and last.end == start:  # same process continues
            last.end = end                                  # -> make the bar longer
        else:
            self.timeline.append(Segment(pid, start, end))
```

🎤 ماذا أقول للدكتورة

الكلاس **Dispatcher** تستخدمه الخوارزميات الخمس كلها لبناء الخط الزمني. أهم دالة فيه **dispatch**: عندما ننتقل من عملية إلى عملية مختلفة يحدث **context switch**، أي أن النظام يحفظ حالة العملية القديمة في الـ PCB الخاص بها ويحمّل حالة الجديدة. أنا أعدّ هذه التبديلات، وإذا اختار المستخدم تكلفة للتبديل أضيف مستطيلاً رمادياً اسمه **CS** في الرسم وأؤخر بداية العملية، وهذا ما يسمى **dispatcher latency**. أما الدالة **add** فإذا استمرت نفس العملية تطيل المستطيل بدل ما ترسم مستطيلات صغيرة كثيرة.

الجزء 3 — مقارنة كل الخوارزميات (F10)

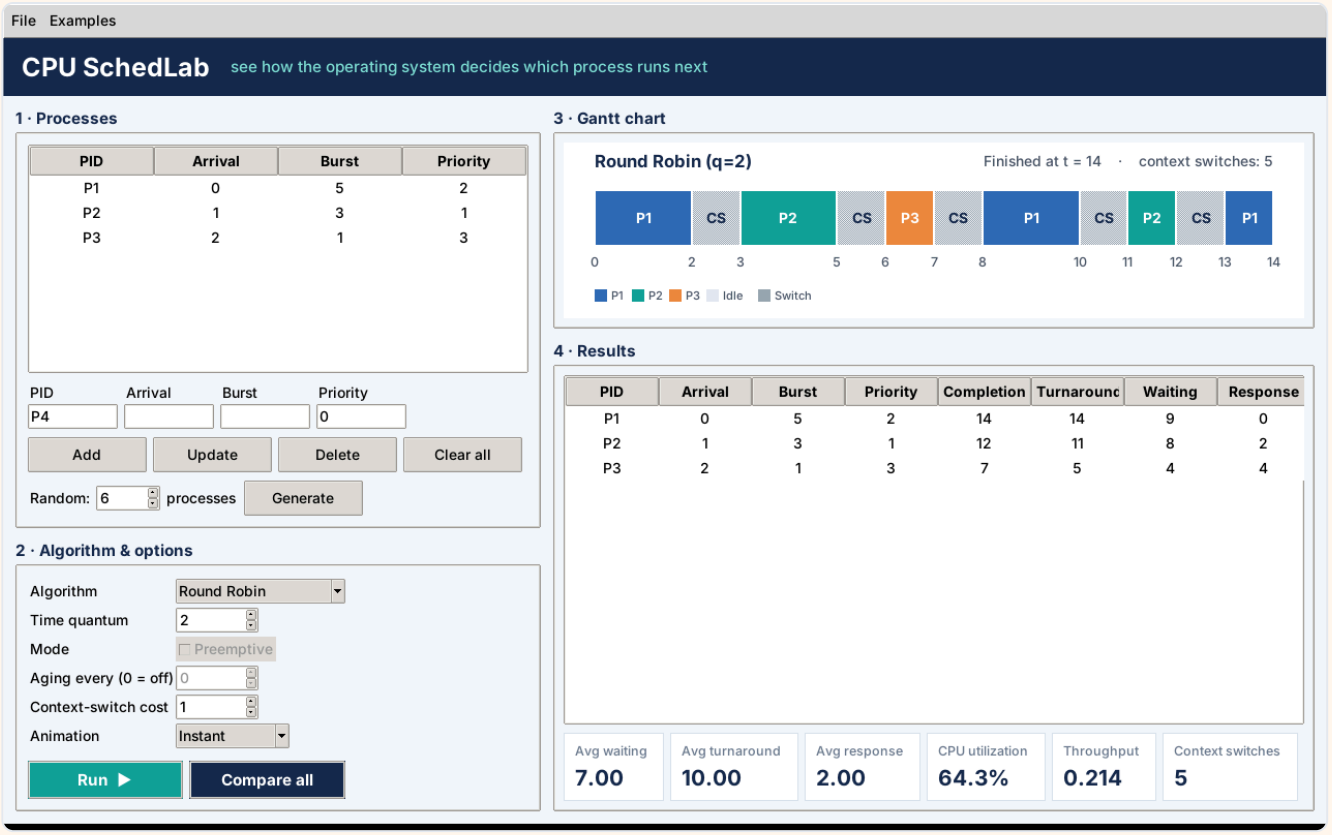
CPU_SchedLab.py lines 237–243

```python
def compare_all(processes, options):
    """F10 Run EVERY algorithm on the SAME processes and collect the averages."""
    summaries = []
    for name in ALGORITHMS:
        _, summary = compute_metrics(run(name, processes, options))
        summaries.append(summary)
    return summaries
```

🎤 ماذا أقول للدكتورة

الدالة **compare_all** هي وظيفة المقارنة: تشغّل الخوارزميات الخمس على **نفس العمليات** وتجمع المتوسطات لكل واحدة، وهذا ما يسمّيه الكتاب **deterministic modeling**. وبعدها الدالة **plot_comparison** ترسم أربعة رسوم أعمدة باستخدام **Matplotlib**: متوسط الانتظار، ومتوسط الـ turnaround، ومتوسط الـ response، وعدد التبديلات، وتضع إطاراً أسود حول الخوارزمية الأفضل في كل رسم.

🖥️ ماذا أعرض على الشاشة



أختار Round Robin مع quantum = 2 وتكلفة تبديل = 1 وأضغط Run. تظهر مستطيلات **CS** الرمادية: صار عندنا 5 تبديلات، والوقت الكلي 14 بدل 9، ونسبة استخدام المعالج نزلت إلى **64%**. هذا يوضح أن الـ context switch وقت ضائع.

🖥️ ماذا أعرض على الشاشة



أضغط **Compare all** فتظهر نافذة المقارنة، والنجمة ★ على الخوارزمية صاحبة أقل متوسط انتظار، وهي SRTF في مثالنا. ونلاحظ أن Round Robin ليست الأفضل في الانتظار، لكنها جيدة في الـ **response time**، ولذلك تناسب الأنظمة التفاعلية.

4 عروب أحمد العمرو

F7 · F9 · المقاييس · Priority scheduling

🎤 البداية — أعرّف بنفسي

السلام عليكم دكتورة، أنا عروب العمرو، ومسؤولة عن خوارزمية **Priority** بوضعيها ومع الـ **aging**، وعن حساب **مقاييس الأداء** لكل الخوارزميات.

الجزء 1 — خوارزمية Priority مع الـ Aging (F7)

CPU_SchedLab.py — lines 275–307

```python
def priority_schedule(processes, options):
    """F7 Priority: smallest priority number runs first. Aging prevents starvation."""
    cpu = Dispatcher(options.switch_cost)
    remaining = {p.pid: p.burst for p in processes}
    current = {p.pid: p.priority for p in processes}       # priority after aging
    waited = {p.pid: 0 for p in processes}
    time, running = 0, None
    while any(remaining.values()):
        ready = [p for p in processes if p.arrival <= time and remaining[p.pid] > 0]
        if not ready:
            next_arrival = min(p.arrival for p in processes if remaining[p.pid] > 0)
            cpu.idle(time, next_arrival)
            time = next_arrival
            continue
        if running and remaining[running] > 0 and not options.preemptive:
            p = next(q for q in ready if q.pid == running)  # non-preemptive: keep it
        else:                                               # best priority wins
            p = min(ready, key=lambda q: (current[q.pid], q.pid != running, q.arrival))
        start = cpu.dispatch(p.pid, time)
        cpu.run(p.pid, start, start + 1)
        elapsed, time = start + 1 - time, start + 1
        remaining[p.pid] -= 1
        running = p.pid
        waited[p.pid] = 0
        if options.aging > 0:                               # AGING
            for q in ready:
                if q.pid != p.pid:
                    waited[q.pid] += elapsed
                    while waited[q.pid] >= options.aging:   # waited long enough ->
                        waited[q.pid] -= options.aging
                        current[q.pid] = max(0, current[q.pid] - 1)   # more important
    mode = "preemptive" if options.preemptive else "non-preemptive"
    return cpu.result(f"Priority ({mode}{', aging' if options.aging else ''})", processes)
```

🎤 ماذا أقول للدكتورة

في جدولة الأولوية تشتغل العملية الأهم أولاً، و**الرقم الأصغر يعني أولوية أعلى**. القاموس **current** يحفظ الأولوية الحالية لكل عملية لأنها قد تتغير بالـ aging. في كل وحدة زمنية: إذا كان الوضع **non-preemptive** والعملية الحالية لم تنتهِ فإنها تكمل. وإلا أختار صاحبة أصغر رقم أولوية، وبهذا في الوضع **preemptive** تستطيع عملية أهم أن تقاطع العملية الحالية. المشكلة هنا هي الـ **starvation**: عملية منخفضة الأولوية قد تنتظر للأبد إذا استمرت العمليات المهمة بالوصول. والحل هو الـ **aging** في الجزء الأخير: كل ما تنتظر العملية عدداً من الوحدات يساوي aging، ينقص رقم أولويتها بواحد حتى يصل إلى صفر. فكلما طال انتظارها أصبحت أهم، ولا يمكن أن تنتظر للأبد. وجرّبنا مثال الكتاب فكان متوسط الانتظار **8.2** مثل الكتاب.

الجزء 2 — مقاييس الأداء (F9)

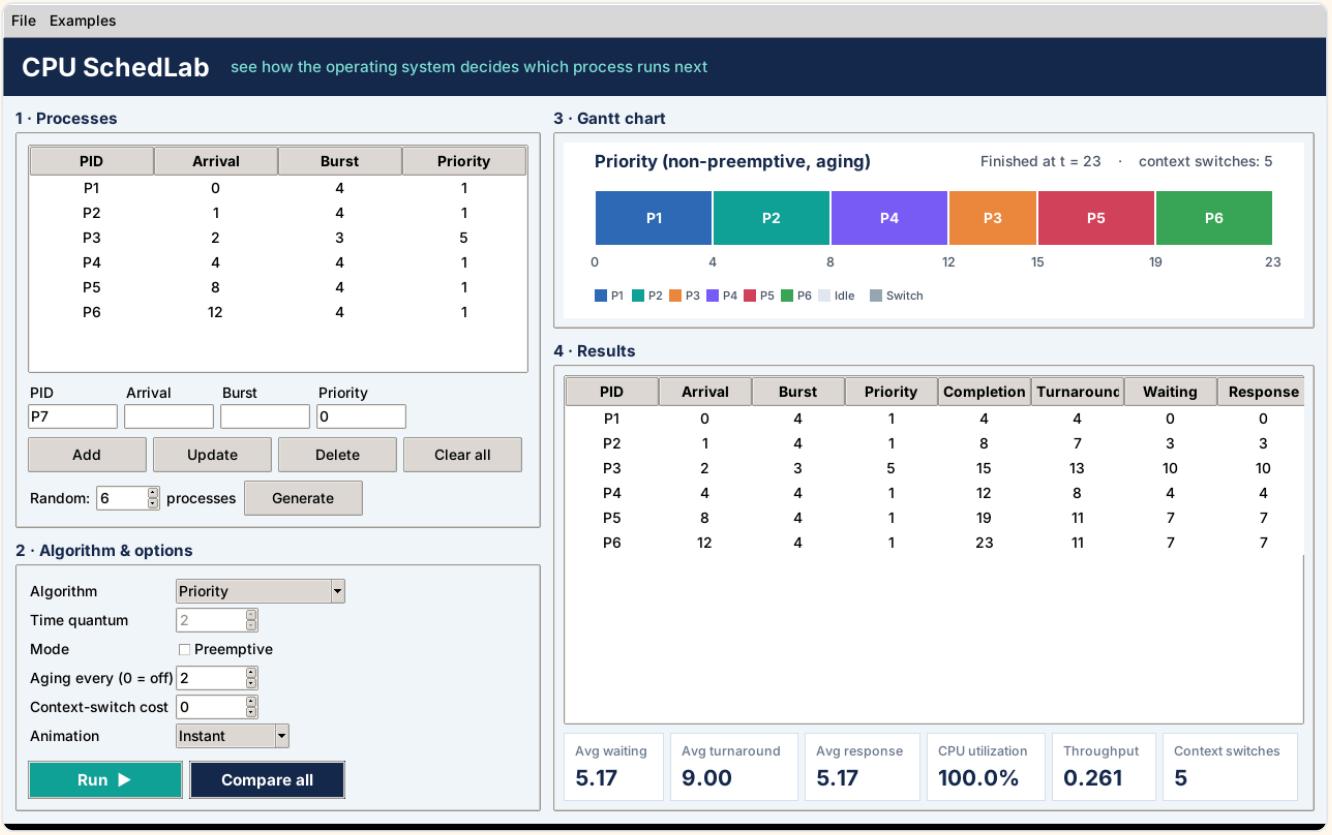
CPU_SchedLab.py lines 310–334

```python
def compute_metrics(result):
    """F9 Waiting, turnaround and response time per process + averages."""
    rows = []
    for p in result.processes:
        blocks = [s for s in result.timeline if s.pid == p.pid]
        completion = blocks[-1].end                        # end of its LAST block
        first_run = blocks[0].start                        # start of its FIRST block
        turnaround = completion - p.arrival
        waiting = turnaround - p.burst
        response = first_run - p.arrival
        rows.append(dict(pid=p.pid, arrival=p.arrival, burst=p.burst, priority=p.priority,
                         completion=completion, turnaround=turnaround,
                         waiting=waiting, response=response))
    n, total = len(rows), result.end_time
    busy = sum(s.end - s.start for s in result.timeline if s.pid not in (IDLE, SWITCH))
    summary = dict(
        algorithm=result.algorithm,
        avg_waiting=sum(r["waiting"] for r in rows) / n,
        avg_turnaround=sum(r["turnaround"] for r in rows) / n,
        avg_response=sum(r["response"] for r in rows) / n,
        cpu_utilization=100 * busy / total,                # % of time doing useful work
        throughput=n / total,                              # processes per time unit
        switches=result.switches,
    )
    return rows, summary
```

🎤 ماذا أقول للدكتورة

الدالة **compute_metrics** تحسب معايير الجدولة. لكل عملية آخذ من الخط الزمني نهاية آخر مستطيل لها، وهو وقت الانتهاء **completion**، وبداية أول مستطيل، وهو أول تشغيل. ثم أطبّق قوانين الكتاب: **Turnaround = Completion − Arrival**، و**Waiting = Turnaround − Burst**، و**Response = First run − Arrival**. وللمجموعة كاملة أحسب المتوسطات، ونسبة استخدام المعالج **CPU utilization** وهي الوقت المفيد مقسوماً على الوقت الكلي بدون الـ idle والـ CS، والـ **Throughput** وهو عدد العمليات مقسوماً على الوقت الكلي. مثال SRTF: انتظار P1 = 9 − 0 − 5 = 4، وP2 = 5 − 1 − 3 = 1، وP3 = 3 − 2 − 1 = 0، فالمتوسط **1.67**.

🖥️ ماذا أعرض على الشاشة



من قائمة **Examples** أختار مثال Starvation. العملية P3 أولويتها 5 (منخفضة). بدون aging تنتهي آخر واحدة عند الزمن **23**. وعندما أضع aging كل وحدتين تصبح أهم مع الانتظار، فتشتغل عند 12 وتنتهي عند **15**، كما في الصورة. وأفعّل Preemptive لأعرض المقاطعة.

5 سارة محمد المجالي

الواجهة الرسومية والرسم · F12 · F8

🎤 البداية — أعرّف بنفسي

السلام عليكم دكتورة، أنا سارة المجالي، ومسؤولة عن الـ **Gantt chart** بنسختيه البسيطة والمتحركة، وعن **الواجهة الرسومية** كاملة، وعن **حفظ وتحميل وتصدير** الملفات.

الجزء 1 — الـ Gantt chart البسيط (F8 · المرحلة 2)

CPU_SchedLab.py — lines 375–382

```
375  def text_gantt(timeline):
376      """F8 (Phase 2) the SIMPLE Gantt chart as text:  | P1 | P2 |  then the times."""
377      bar, axis = "|", "0"
378      for s in timeline:
379          width = max(len(s.pid) + 2, (s.end - s.start) * 2)
380          bar += ("--" if s.pid == IDLE else s.pid).center(width) + "|"
381          axis += str(s.end).rjust(width + 1)
382      return bar + "\n" + axis
```

🎤 ماذا أقول للدكتورة

هذه النسخة الأولى من الـ **Gantt chart**. الـ Gantt chart يرسم الخط الزمني للمعالج: كل مستطيل يمثل عملية، وطوله يساوي مدة تشغيلها. الدالة **text_gantt** تمرّ على مقاطع الخط الزمني، ولكل مقطع تكتب اسم العملية بعرض يتناسب مع مدتها، وتكتب تحته وقت النهاية. وعند الضغط على Run يُطبع هذا الرسم في الطرفية، مثلاً: | P3 | P2 | P1 | وتحته الأوقات 0 5 8 9.

الجزء 2 — الرسم المتحرك (F8 · المرحلة 3)

CPU_SchedLab.py — lines 402–415

```
402      def play(self, result, seconds):
403          """Draw the chart little by little: 40 frames per second, `seconds` long."""
404          if self.job:
405              self.after_cancel(self.job)
406          self.result, end = result, result.end_time
407          self.now = 0.0 if seconds > 0 else end
408          step = end / max(1, seconds * 40)
409
410          def tick():
411              self.now = min(end, self.now + step)
412              self.redraw()
413              if self.now < end:
414                  self.job = self.after(25, tick)             # next frame (window stays free)
415          tick()
```

🎤 ماذا أقول للدكتورة

هذا الكلاس **GanttCanvas** يرسم الـ Gantt **بشكل متحرك**. الدالة **play** تقسم الوقت الكلي على عدد من الإطارات، بمعدل 40 إطاراً في الثانية، وكل 25 جزءاً من الألف من الثانية تزيد المتغير **now** وتعيد الرسم. ودالة الرسم **redraw** ترسم فقط الجزء الذي قبل **now**، وترسم خطاً أحمر عند الوقت الحالي، وتكتب في الأعلى أي عملية على المعالج الآن. استخدمت **after** بدل **sleep** حتى لا تتجمد النافذة أثناء الحركة. ولكل عملية لون ثابت، والـ idle رمادي فاتح، والـ context switch مخطط.

الجزء 3 — زر Run: الواجهة لا تحسب شيئاً بنفسها

CPU_SchedLab.py lines 725–742

```python
    def on_run(self):
        """The Run button: core -> metrics -> draw. The GUI calculates nothing itself."""
        try:
            result = run(self.algorithm.get(), self.processes, self.read_options())
        except ValueError as err:
            return messagebox.showerror("Cannot run", str(err))
        rows, summary = compute_metrics(result)
        self.last = (result, rows, summary)
        self.gantt.play(result, SPEEDS[self.speed.get()])
        print(result.algorithm, "\n" + text_gantt(result.timeline), "\n")   # simple Gantt
        self.rtable.delete(*self.rtable.get_children())
        for r in rows:
            self.rtable.insert("", "end", values=list(r.values()))
        for key, label in self.cards.items():
            v = summary[key]
            label.config(text=f"{v:.1f}%" if key == "cpu_utilization" else
                         f"{v:.3f}" if key == "throughput" else
                         f"{v:.2f}" if isinstance(v, float) else str(v))
```

🎤 ماذا أقول للدكتورة

هذه الدالة تعمل عند الضغط على زر **Run**، وهي توضح أن الواجهة **لا تحسب شيئاً بنفسها**. تقرأ اختيار المستخدم، ثم تستدعي **run** من القلب لتشغيل الخوارزمية، ثم **compute_metrics** لحساب المقاييس، وبعدها ترسم الـ Gantt المتحرك وتملأ جدول النتائج وبطاقات المتوسطات. وإذا كان هناك خطأ في الإدخال، مثل burst يساوي صفراً أو حرف بدل رقم، تظهر رسالة خطأ واضحة بدل ما يتوقف البرنامج.

الجزء 4 — حفظ وتحميل وتصدير CSV (F12)

CPU_SchedLab.py lines 466–491

```python
def save_workload(path, processes):
    """F12 Save the processes in a CSV file (opens in Excel)."""
    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["pid", "arrival", "burst", "priority"])
        for p in processes:
            writer.writerow([p.pid, p.arrival, p.burst, p.priority])

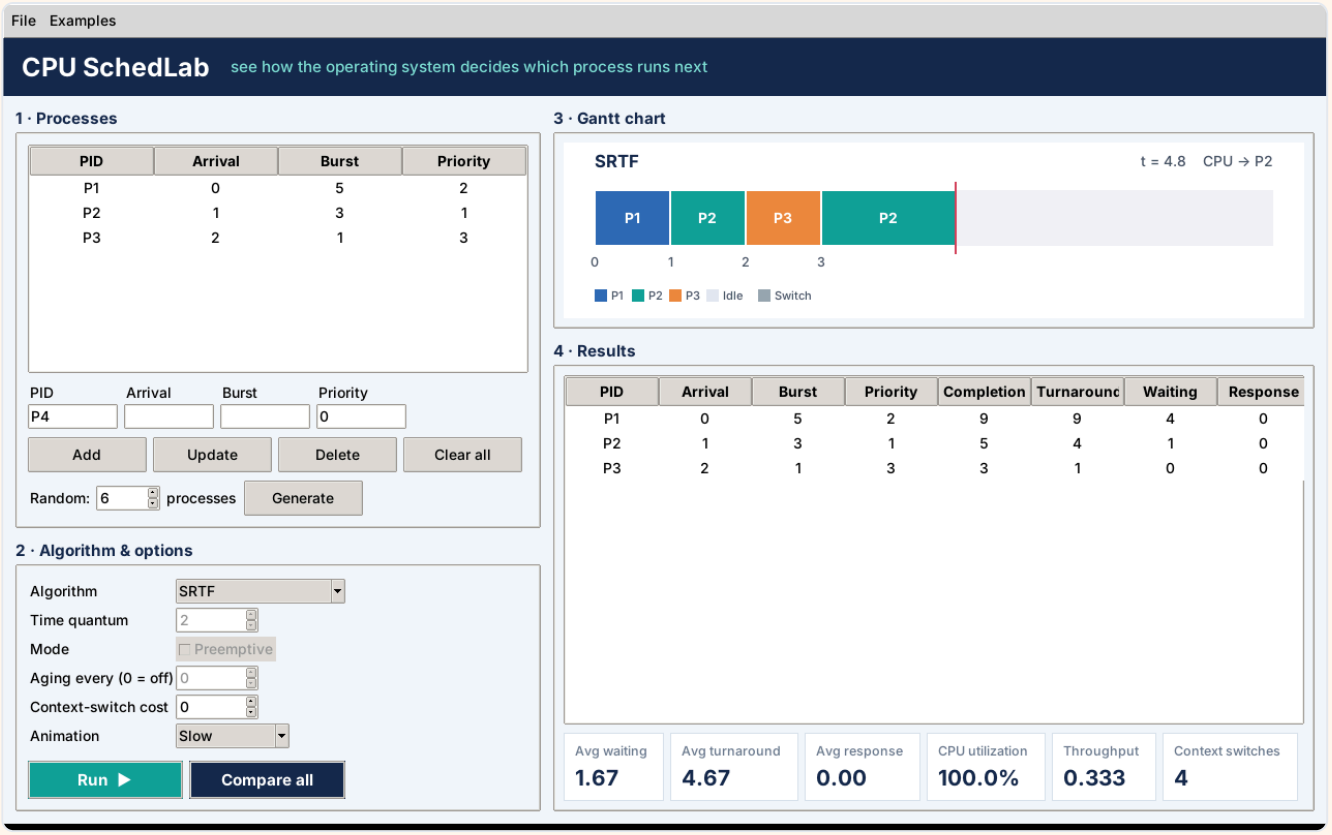

def load_workload(path):
    """F12 Read processes back from a CSV file."""
    with open(path, newline="", encoding="utf-8") as f:
        return [make_process(r["pid"], r["arrival"], r["burst"], r.get("priority") or "0")
                for r in csv.DictReader(f)]


def export_results(path, result, rows, summary):
    """F12 Export the Gantt timeline, the results table and the averages as CSV."""
    with open(path, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["Algorithm", result.algorithm])
        w.writerow(["pid", "start", "end"])
        w.writerows([s.pid, s.start, s.end] for s in result.timeline)
        w.writerow(list(rows[0].keys()))
        w.writerows(list(r.values()) for r in rows)
        w.writerows([k, round(v, 3) if isinstance(v, float) else v] for k, v in summary.items())
```

🎤 ماذا أقول للدكتورة

هذه وظيفة **F12** وتمثّل مفهوم عمليات **نظام الملفات**. الدالة **save_workload** تحفظ العمليات في ملف **CSV**، وهو جدول نصي يفتح في Excel. و**load_workload** تقرأ الملف وترجع العمليات، وتمرّ كل قيمة على make_process حتى نتأكد أن الملف صحيح. أما **export_results** فتصدّر الخط الزمني وجدول النتائج والمتوسطات في ملف واحد. وكل هذا موجود في قائمة **File** في الواجهة.

🖥️ ماذا أعرض على الشاشة



أعرض النافذة وأقسامها الأربعة: 1) جدول العمليات مع Add و Update و Delete و Generate، 2) اختيار الخوارزمية وخياراتها، وهي تتفعّل حسب الخوارزمية، 3) الـ Gantt المتحرك، 4) النتائج. أضغط Run بسرعة Slow حتى تظهر الحركة والخط الأحمر، ثم أضغط على الرسم ليكتمل فوراً. ثم أجرّب إدخالاً خاطئاً لأُظهر رسالة الخطأ.